MLOps Pipeline Development: The OSSARA Use Case

Sergio Moreschini
Tampere University
Tampere, Finland
sergio.moreschini@tuni.fi

David Hästbacka
Tampere University
Tampere, Finland
david.hastbacka@tuni.fi

Davide Taibi
University of Oulu,
Tampere University
Oulu, Finland
Tampere, Finland
davide.taibi@oulu.fi

ABSTRACT

Software development based on MLOps practices is entering its early adoption stage. As for it, practitioners and researchers are starting to develop pipelines composed of tools capable of automating the whole software lifecycle. The development of the pipeline however is not as straightforward as it looks and some key points need to be addressed. In this work, we propose our vision for the development of the MLOps pipeline by showing what to keep into account when choosing the tools for each step of a pipeline. The proposition has been backed up by describing a developed use case scenario: the OSSARA use case. We believe that the presented use case, as well as the remarks presented for the process of tool selection for each MLOps phase, will help practitioners and researchers in the process of developing their own pipelines.

CCS CONCEPTS

• **Computer systems organization** → **Embedded systems**; *Redundancy*; Robotics; • **Networks** → Network reliability.

KEYWORDS

MLOps, Machine Learning, pipelines

ACM Reference Format:
Sergio Moreschini, David Hästbacka, and Davide Taibi. 2023. MLOps Pipeline Development: The OSSARA Use Case. In *International Conference on Research in Adaptive and Convergent Systems (RACS '23), August 6–10, 2023, Gdansk, Poland*. ACM, New York, NY, USA, Article 4, 8 pages. https://doi.org/10.1145/3599957.3606211

1 INTRODUCTION

The increased adoption of Machine Learning (ML)-based software has inevitably created a need for a new method to efficiently develop software based on this new trend. Even if the term MLOps (i.e. Machine Learning Operations) has been widely accepted in literature, the concept does not provide uniform guidelines to develop ML-based software.

As a direct evolution of DevOps, it inherits some of its fundamental concepts such as Continuous Integration (CI), Continuous Delivery (CD), and the automation derived from such concepts. Some of these concepts take a step further compared to DevOps by adding, for example, new levels of automation such as Continuous Training of the underlying ML code behind the software.

The best way to achieve complete automation in MLOps is to create pipelines capable of automating the process of developing, testing, deploying, and monitoring ML models at scales. Such pipelines are created to meet the goals set by MLOps of ensuring the quick delivery of high-quality models as well as the cooperation among software and ML developers on one side, and IT operations on the other.

In order to optimize the process of MLOps pipeline development it is essential to have a full understanding of both the underlying ML code and the results that need to be achieved. To do so it is also essential to have the right set of tools to ensure full compatibility among the different MLOps phases covered.

In this work, we aim at providing some insights related to the process of choosing and combining tools for creating MLOps pipelines. In particular, we want to share our experience when developing an MLOps pipeline based on a model for OSS Abandonment Risk Assessment, namely OSSARA.

The work provides a different perspective on the development of a software-based MLOps pipeline. Contrary to most works that focus on a definition of MLOps based on the different levels of automation provided in [2], we focus our attention on reducing the gap between DevOps and MLOps by considering the latter as a natural evolution. A starting point is the selection of different open source tools based on the infinite MLOps pipeline as presented in [13]. The selection of such tools is also done taking into account the minimum number of usable OSS tools to create a meaningful pipeline for the tested use case.

Specifically, we aim to create a system that can perform continuous training on an ever-increasing dataset measured with different metrics and trained with different models, as we develop an MLOps pipeline for OSS abandonment risk assessment. The developed system will also need to be able to select the model with the best result and perform an automatic deployment of a new release on the basis of such a model.

The contribution of this paper is two-fold:

- **MLOps Pipeline Creation:** We provide insight into the process of developing an MLOps pipeline by focusing on the different stages of the infinite MLOps pipeline, which is derived from the original infinite DevOps pipeline.
- **Use case development:** We test the theorized pipeline by developing a specific use case based on OSSARA, the OSS Abandonment Risk Assessment model. Allowing the model to perform continuous training and deployment through REST API.

This work is licensed under a Creative Commons Attribution International 4.0 License.

RACS '23, August 6–10, 2023, Gdansk, Poland
© 2023 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0228-0/23/08.
https://doi.org/10.1145/3599957.3606211

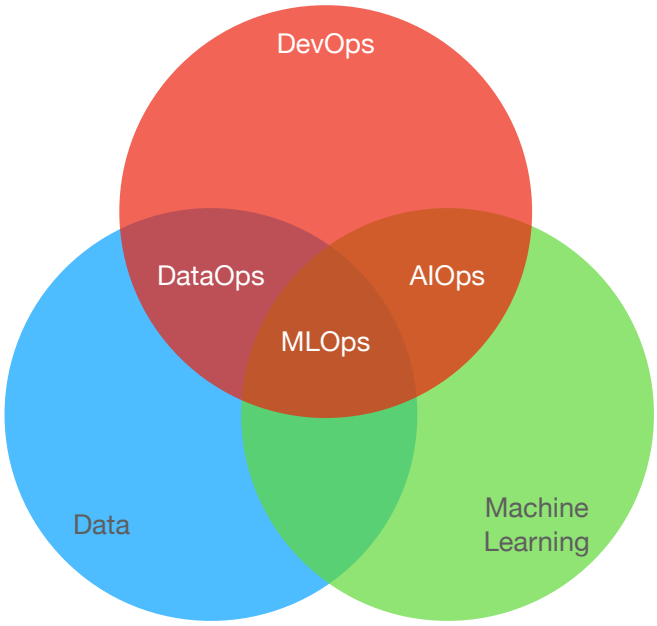

Figure 1: Different branches derived from DevOps

The results of this work will enable researchers and practitioners to gain insight into the process of developing fully automated ML-based software by making use of MLOps guidelines. In particular, we aim at increasing awareness of the importance of performing data versioning when developing ML-based software. On the other hand, the newly developed OSSARA model will achieve continuous updates through continuous training and deployment.

The rest of the paper is structured as follows. Section 2 covers the background starting from DevOps and reaching different branches such as MLOps. Section 3 presents the related works. Section 4 describes the process of selecting the most appropriate tool for each stage of the MLOps infinite loop. Section 5 introduces the original OSSARA model and describes how the risk assessment is calculated. Section 6 describes the process of creating a pipeline based on the OSSARA model. Section 7 provides some discussions while Section 7 concludes the work.

2 BACKGROUND

Since the dawn of software development, software engineers have been developing methodologies and guidelines to make the process of developing software a systematic process capable of ensuring specific levels of quality. The guidelines defined in the Software Development Lifecycle (SDLC) have been providing the correct methodology to follow for around 50 years.

With the evolution of the traditional software development model, the two main categories responsible for creating and maintaining the software, namely Developers and IT Operations necessitated a new set of practices aiming at breaking down the borders between such categories and boosting the collaboration among such entities. Such a set of practices has been defined as DevOps, a simple term coined based on the names of the two main leading actors responsible for such a revolution.

Even if there is no clear and uniform definition for DevOps has been set [9], all the definitions agree on 3 particular points:

(1) **What** - DevOps is a set of practices
(2) **Who** - combining software development and IT operations
(3) **Goal** - to deliver software quickly, rapidly, and efficiently.

Nowadays DevOps evolved by acquiring some of the main principles of other practices such as Agile development and CI/CD. From a different perspective, however, practitioners and technology experts focus more on the tools that allow providing the automation and the goals set by DevOps than the set of practices defined by DevOps itself. A clear example of such a shift is the periodic table of DevOps Tools a famous tool developed to identify the best tools for the software delivery lifecycle [4].

Different evolution of DevOps have been proposed and are receiving increasing attention. These new sets of practices are combining DevOps with the different entities on which is posed the main focus. A representation of DevOps and its branches is depicted in Figure 1. AIOps, for example, is defined as artificial intelligence for IT Operations and, even if places more attention on the Ops phase it is usually categorized as a branch of DevOps [6].

Among the different branches of practices derived from DevOps, we have the one that is focused on data: DataOps. In [12] Mainali et al. analyzed the different definitions of the term DataOps starting from what has been traced as the original definition in a blog post on the IBM Big Data & Analytics Hub titled "3 reasons why DataOps is essential for big data success[1]", which highlights the importance of executing data analytics rapidly. They also analyzed different works that focused on different perspectives of the topic dividing these into 3 main categories: goal-oriented, activities-oriented, and process and team-oriented. From the goal-oriented perspective, DataOps is mostly identified as a process (not a set of practices) to "eliminate errors and inefficiency in data management", from the activities perspective it is a set of practices in the data analytics field that combines DevOps and Agile methodologies. At last from the process and team-oriented perspective, it is defined as an approach for managing activities of the data lifecycle which requires collaboration among data creators and consumers.

A definition that seems to include all the previous definitions is the one presented in [5] by Ereth which defined DataOps as *"set of practices, processes, and technologies that combines an integrated and process-oriented perspective on data with automation and methods from agile software engineering to improve quality, speed, and collaboration and promote a culture of continuous improvement"*.

All of the previous definitions do not take into account what has been made on the data, for this reason, it has been necessary to define another set of practices that are focused on data-driven applications, namely MLOps. Multiple entities have been trying to provide their vision on what are the most important stages and phases of MLOps, notably the most famous is the one from Google [2]. In the latter, the main focus has been posed on how to implement Continuous Integration (CI), Continuous Delivery (CD), and Continuous Training (CT) for ML systems. In particular, MLOps has been categorized based on the different levels of automation from 0 to 2, which respectively stand for Manual Process and CI/CD automation. An *"MLOps level 2: CI/CD pipeline automation"*

[1]Not available at the time of writing this work

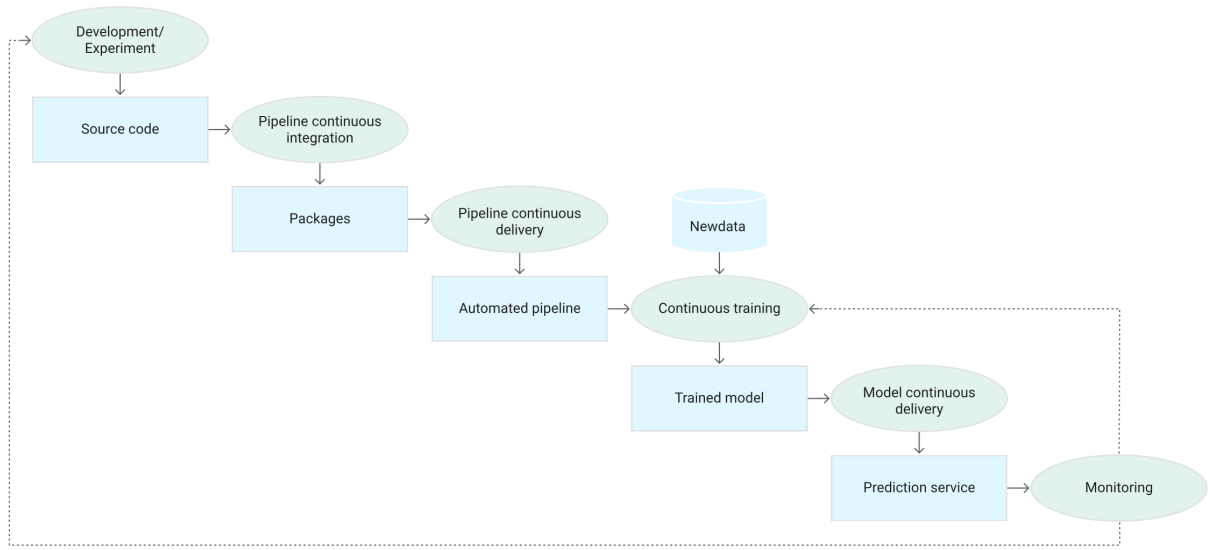

Figure 2: Stages of CI/CD automated pipeline. Reprinted with permission from [2]. (CC BY 4.0).

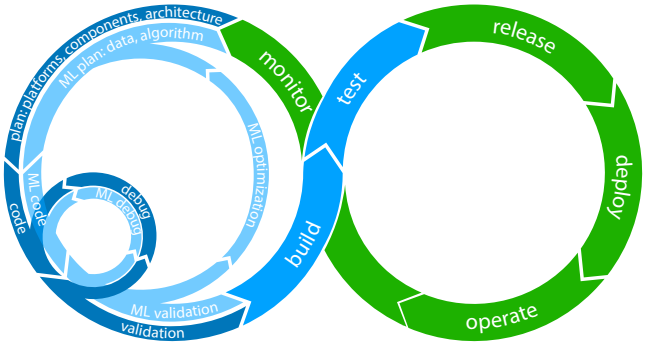

Figure 3: MLOps pipeline. Reprinted with permission from [13]. ©IEEE 2022.

is therefore capable of providing a system capable of *"deploying an ML pipeline that can automate the retraining and deployment of new models"*. Such a system is described by the pipeline depicted in Figure 2.

On the contrary, in [13] Moreschini et al. provided their vision of MLOps by highlighting the similarities and differences compared with DevOps. As such, the proposed pipeline is an evolution of the well know DevOps infinite loop, and as such it is composed of two main areas the Dev and the Ops (Figure 3). The main difference, in this case, is that the Dev part does not just include the work performed by the software developer but also the one from the ML developer which needs to work together to develop the software that will be passed to the IT Operations for the Ops phases.

Following, in the work [16], the authors provided a follow-up to their previous work by providing a list of tools capable of handling the various phases of the MLOps pipeline. Among the different tools provided, some of these have been categorized also as End-to-End Full-stack MLOps tools as they are capable of handling all the stages of the MLOps pipeline, while others have been categorized by the implementation that they can perform (i.e. CT, CI, CD, and OPS). Therefore, they suggest that it is possible to compose an MLOps pipeline by selecting one tool for each phase of the Dev phase and at least one tool for each phase of the Ops phase.

3 RELATED WORKS

In the last year, various works have tried to define a specific set of tools to define ML pipelines and provide their own examples related to how to use such pipelines in specific use cases. In [19], with the aim of presenting the potential of MLOps, the authors compared a specific set of tools for MLOps. After comparing them, in terms of preprocessing, training, management, deployment, and operations in ML projects, they presented an example of a specific workflow for "*Partially Automating Object Detection with MLOps Tools*". The paper bases its definition of MLOps on the different levels from [2], and the example aims to describe a level-1 automation using tools such as Git, GitHub actions, DVC, Hyperopt, and MLflow.

In [23], the authors combined the power of an end-to-end MLOps tool such as Kubeflow with additional tools such as Gitea and Drone to create an ML platform with a DevOps-capable framework. The framework was designed to continuously train several well-known deep learning neural networks, such as ResNet, MobileNet, and Inception V3, specifically to test the consumption in terms of energy and time.

In [18] an Edge MLOps framework has been presented. The framework uses a continuum-like system, as the CT phase is performed in the cloud through a cloud orchestration layer, while inference is performed on an edge device. The framework has been developed with Azure for ops-side management and Docker for deployment. The framework has been tested in a real-world scenario, where the goal was to deploy ML models to predict the air quality of rooms directly on the edge.

In [21], after reviewing the state of the art in MLOps and introducing MLOps as an evolution of DevOps principles, a use case was presented where modular pipelines were built for an ML time series forecasting system. Three approaches for the use cases were presented where an MLOps level 2 automated CI/CD pipeline is implemented [2]. Some of the tools that have been used are GIT, Jenkins, Docker, MLflow and Django.

4 DEVELOPING AN MLOPS PIPELINE

The process of creating an MLOps pipeline provides a very different approach compared to basic ML applications.

Plan: In this case, the selection of the tools used to perform the whole process is a vital step that needs to be taken into account already in the planning phase and, as part of the planning phase, it might variate along the development process.

Code/ML: Following the coding/ML phase in most cases is not usually associated with specific tools as the IDEs are only supporting the coding phase. Lately, there is an increased interest in different approaches to ML such as automated ML which is provided in some end-to-end MLOps tools such as Microsoft Azure Machine Learning, which automates the process of building machine learning models. For this reason, they can be defined as an extension of the library commonly used for creating ML architectures.

Build: For what concern the Build we find the biggest differences as we are not going to track the codes anymore but also its inputs and outputs. A huge part of creating an MLOps pipeline is to keep track of what are the different inputs associated with the various versions of the code and what are the outputs generated for such combination. In this work, this has been done by making use of the high compatibility between DagsHub [3] and DVC [8] for the input phase and between DagsHub and MLflow [17] for the output phase.

The importance of Data tracking is the fundamental part of DataOps and such similarity makes it often hard to delineate the difference between DataOps and MLOps. In [22], Tamburri indicates the *operation of Machine Learning continuously* as the borderline between DataOps and MLOps, specifically, MLOps needs to be composed of 5 specific functions:

(1) **Data ingestion/transport**: The layer at which the source data is collected on a regular basis and sent to the data warehouse or other such database, or ingested into the next layer.
(2) **Data transformation**: The layer that contains a set of transformations that support learning algorithms. It can include both preprocessing and data augmentation.
(3) **Continuous (re)Training** The layer responsible for training the network at regular intervals or when a certain event occurs.
(4) **Continuous Deployment** The layer responsible for deploying the system whenever the results achieved from the training phase overcome the previous training.
(5) **Output Presentation.** The layer responsible for constant monitoring of the deployed system and for presenting the desired results to the end-users.

This means that while both DataOps and MLOps are based on fundamentals of collaborative workflow, automation, and standardization, the main difference is that while it is possible to have DataOps without Machine Learning (and therefore MLOps) it is not possible to have the contrary as the second one relies on the first.

Test: The test phase is a fundamental step in the DevOps pipeline and is somehow a very ad hoc stage for what concerns MLOps as it is mostly dependent on the application that is tried to produce. First of all, when talking about ML, and mentioning test the main concept that comes to mind is to test how well the trained model performs on a dataset which has never been seen on the contrary when developing software the concept of testing is related mostly to ensuring the built software is defect free [10]. For its nature is therefore important to use tools to ensure the correct behavior of the system before encapsulating it and deploying it. From the algorithm development (ML) point of view tools such as DeepChecks [1] provide a continuous validation testbed to the ML system. On the other side, from the deployment (software) point of view, it might be necessary to run further tests due to the software nature of the system and the tools selected from the deployment stage. When choosing a tool for ML deployment it is quite normal to expect the integration of tools used to perform testing before deployment. However, extra testing can be performed by using libraries dedicated to it (for example PyTest in Python).

5 OSSARA

The OSS Abandonment Risk Assessment (OSSARA) has been first presented in [11]. The OSSARA model has been developed to provide a valuable tool to increase awareness of the problem of software abandonment. Such a problem is of particular interest in Open Source Software (OSS) components and libraries because such tools are developed in a collaborative, public manner.

Nowadays the process of composing software based on the adoption of *Components Off The Shelf* (COTS) is an effective method for software development. As OSS can be viewed as COTS, therefore, new software can be developed by creating custom code that aggregates different OSSs. In this particular environment, it is of vital importance that all the components of the software are reliable and stable, especially because the abandonment of OSS components could provide partial or full instability of the complete software. Among the main problems generated by the abandonment of an OSS there are security-related issues that might expose the users and the company creating the software.

The OSSARA model is calculated on 2 main factors:

(1) the likelihood of each component losing maintenance support,
(2) the importance of each component in the main system.

This means that starting from a system composed of several OSS components, it is analyzed the composition of the system to retrieve the weight of the component and, for each of these components, the abandonment probability is calculated in a specific time-frame (Figure 4). Formally this can be calculated as:

$$R_a = \sum_{m=1}^{k} w(O_m) * r(O_m) \tag{1}$$

where R_a is the overall abandonment risk of the system, $w(O_m)$ is the weight of the OSS component O_m and $r(O_m)$ is the risk that O_m will be abandoned. The method has been validated by evaluating

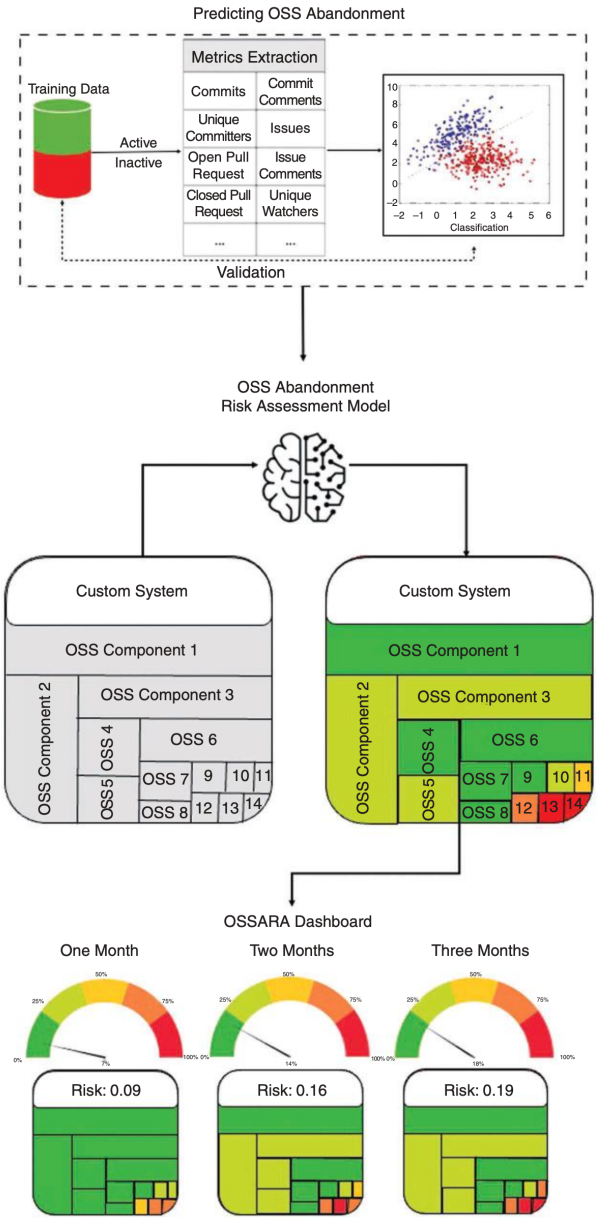

Figure 4: The OSSARA process. Reprinted with permission from [11]. (CC BY 4.0).

12,208 OSS projects that contain at least 1,000 commits from at least 5 unique contributors and are watched by at least 100 users. The classification has been performed using four different classification algorithms: decision tree, support vector machine (SVM), logistic regression, and naive Bayes. The results have shown that among these the logistic regression was providing the best results.

6 IMPLEMENTING THE OSSARA USE CASE

The OSSARA method has been used as a use case to develop an MLOps system. Following the list of tools suggested in [16], we created an MLOps pipeline to develop the OSSARA system in MLOps using Open Source Tools. In particular, such an MLOps system performs continuous training (CT) to compute the risk factor of each tool in the OSSARA system.

The implemented system will calculate the risk factors of each tool based on the model that is automatically chosen to be deployed. Such a model will be the one that provides the highest level of F1-score among all the models that have been trained with the different sets of the dataset and with the different classification algorithms.

The reason behind this choice is to simulate a system that is capable of performing continuous training to update its knowledge based on the current state of OSS projects of GitHub.

6.1 Continuous Training

One of the primary outcomes we tried to achieve in this development was the possibility of having a CT setup. To achieve such desideratum, we preprocessed the dataset used in OSSARA (https://doi.org/10.6084/m9.figshare.16944001. v1) so that it was divided into different time frames.

The different data frames have been created so that the starting data frame would include 2 years and half of the data for the first training, 3.5 for the second, and so on, generating in total 18 different sets for training.

The sets would then be loaded by the system sequentially so that it would simulate a continuously increasing dataset.

6.2 Tools Selection

The tools chosen to create this specific pipeline are DagsHub, DVC, and MLflow. For what concerns the developing part of the project DagsHub and DVC can provide all the desiderata. More into detail, even if both of these systems are categorized as *Build* tools, the cooperation of these tools can cover also different phases such as *Planning*.

To start the project, at first, a DagsHub repository has been initialized. Together with this, the DVC storage has been configured following the guidelines provided by the DagsHub repository. As previously specified the planning phase of the MLOps pipeline has been fulfilled by creating a *dvc.yaml* file which would create a data pipeline representation in the DagsHub repository. The generated representation is depicted in Figure 5.

Following the planning phase, we have the Code/ML phase. The code has been developed in Python, specifically using the scikit-learn library [20] in the Pycharm integrated development environment (IDE) [7] which has been connected to the DagsHub repository to simplify the stages of commit and push.

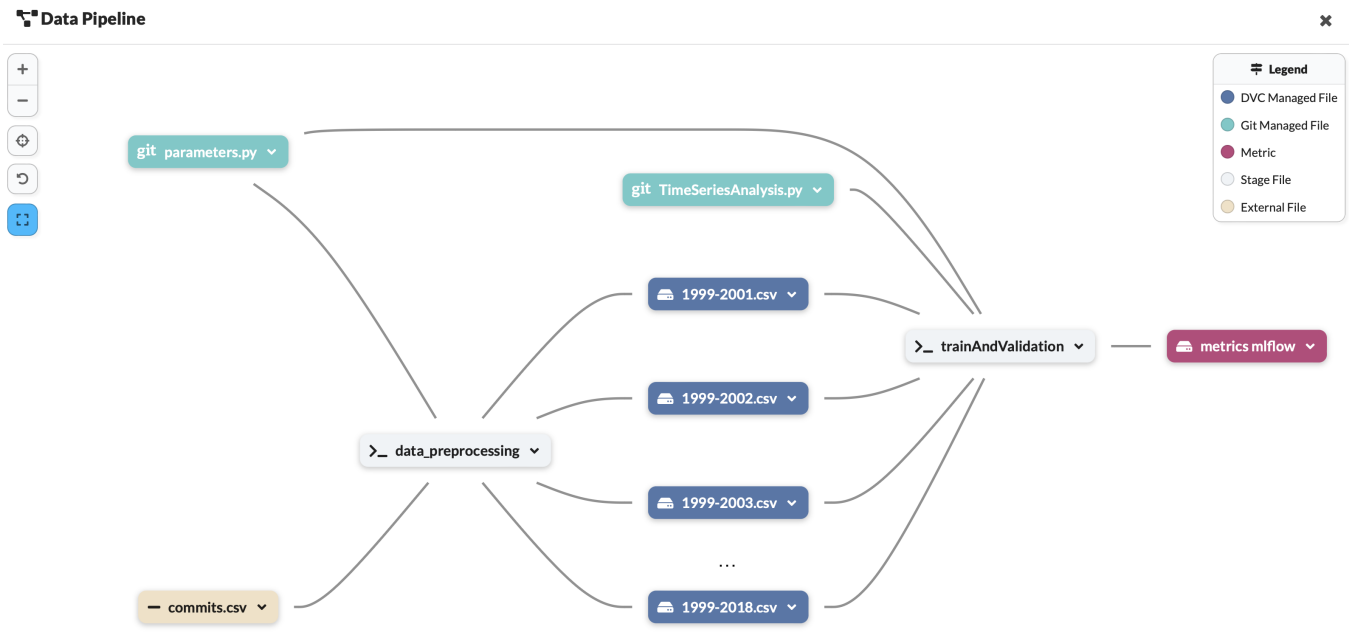

Figure 5: The data pipeline (Planning Phase).

As previously mentioned the phase of Build has been covered by both DagsHub and DVC. As the application does not require anything more than an API, the tools necessary for testing have been provided by the automatic testing performed by the deploying tool before performing the deployment. For what concerns data, as we are using a dataset already used previously we do not need to test it and perform drift analysis.

MLflow has been used to cover all the Operations phases. During the development of the code we integrated MLflow and we ran the experiments by following the MLflow guidelines in the DagsHub repository. Once the experiments have been produced it was possible to lunch the MLflow UI directly from the DagsHub repository, such UI provides information related to the different experiments, including all the metrics extracted (accuracy, macro, active, inactive) the duration, the source from which the experiment has been run and the model associated to the experiment.

A very interesting feature of MLflow is the possibility of deploying the model created as a REST API directly from MLflow, avoiding therefore to use of an external tool to perform such a step. This allows to invoke the local REST API endpoint with a POST input and after running the command it is possible to retrieve the inference results directly in the terminal.

6.3 Training and Deployment

Once the project has been developed, it was possible to perform a complete loop of the MLOps pipeline to measure the timing. A full training step of a single dataset was composed of 4 different algorithms. Each training step was based on the size of the dataset used to perform training. A summary of the timing for each training step has been reported in Table 1.

Once the training has been completed, it was possible to choose a specific run among those. Once the run has been detected, its id has been copied and used to perform a deployment based on REST API. The command used in the terminal was straightforward, as it was necessary to provide the MLflow URI, username, and password as shown in the experiment section of DagsHub together with the command MLFLOW MODELS SERVE –MODEL-URI RUNS:/<*RUN_ID*>/DT –NO-CONDA

The deployment has been immediate (3 seconds in total) and, as no public IP was provided the REST API has been deployed in the localhost port 5000 (as default). To invoke the REST API, a simple curl script with a POST input to the /invocations path was launched with an instant inference response.

6.4 Overall system architecture

The architectural design of the system implementing OSSARA is depicted in Figure 6. The system consists of 3 main components: the online repository, the server and a client.

The online repository, DagsHub, has been connected to DVC and MLFlow for data versioning and model registry.

The server is the one where the code resides, therefore at first the process of coding is completed and the practices of code versioning are finalized by pushing the different versions on DagsHub. Since the model resides online on DagsHub, it can be downloaded whenever it is needed. When the MLOps pipeline loop is started, if the data is not already on the server, it will be downloaded. During

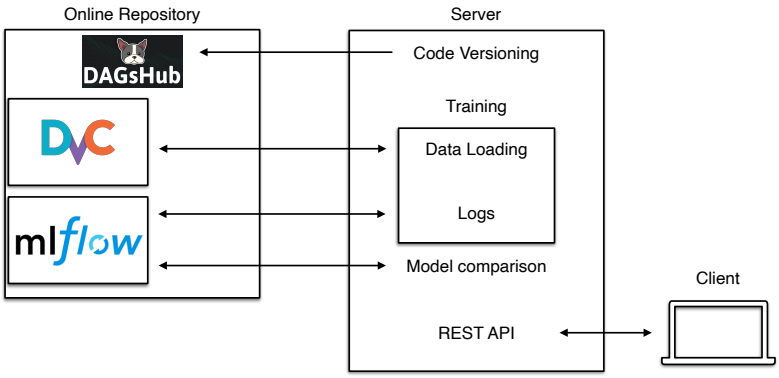

Figure 6: The overall system architecture

Dataset	Decision Tree	SVM	Logistic Regression	Naive Bayes	Total
1999-2001	5.1	4.6	5.6	4.5	**19.8**
1999-2002	4.9	4.7	4.5	5.6	**19.7**
1999-2003	4.9	4.8	4.7	4.5	**18.9**
1999-2004	4.8	5.0	4.6	4.9	**19.3**
1999-2005	4.7	4.8	4.6	4.9	**19.0**
1999-2006	4.7	5.4	4.7	4.7	**19.5**
1999-2007	4.7	4.6	5.3	4.8	**19.4**
1999-2008	4.9	4.8	4.8	4.7	**19.2**
1999-2009	4.9	4.9	4.7	4.8	**19.3**
1999-2010	4.8	4.9	4.7	4.5	**18.9**
1999-2011	4.9	5.2	4.9	4.7	**19.7**
1999-2012	4.9	5.7	4.9	4.8	**20.3**
1999-2013	4.9	6.0	4.7	4.8	**20.4**
1999-2014	5.0	7.2	4.9	4.8	**21.9**
1999-2015	5.1	8.6	6.0	4.4	**24.1**
1999-2016	4.5	10.0	4.8	4.9	**24.2**
1999-2017	5.2	10.0	4.8	4.8	**24.8**
1999-2018	5.1	10.3	4.7	4.6	**24.7**

Table 1: Timing for each training (in seconds).

the training steps, the different training logs are uploaded to the online MLFlow registry connected to DagsHub. Once the training is completed, the different models in the MLFlow registry are compared and once the best one is found, it is deployed through REST API.

Once the model is deployed, a client can now connect to the available port to submit the script and receive the inference from the server.

7 DISCUSSION

For the goal of this project, only part of the OSSARA use case has been developed as we were interested in simulating CT for the ML-based part of the model. The use case provided in the previous section provides an example of an MLOps pipeline based on the use of OSS tools. The use of such tools has been reduced to the minimum in order to provide a simple yet complete MLOps pipeline. It is important to specify that the current state of tools development is heavily based on the application that is intended to develop, and therefore smaller pipeline could be developed for different applications. An example is Computer Vision applications which nowadays have been heavily based on ML (and in particular deep learning) architecture and therefore have received ad-hoc tools such as Picselia or Roboflow.

A very important point that we would like to highlight by developing this work is the centrality of data in MLOps. This is because as the software to develop is heavily dependent on data it is fundamental to track any changes related to the input. For this reason, we see MLOps as a natural evolution of both DevOps and DataOps.

Thanks to this process it is now possible to automatically train and deploy a system capable of continuously monitoring the risk of embedding OSS components as COTS. We believe that such a process is of high interest for companies and developers who need to rely on OSS libraries and tools for developing their own software.

The system could be further implemented in the future by adding an alert system that could trigger messages for specific OSS tools or libraries inserted in a watch list to inform the users of the increased risk of abandonment for the element.

7.1 Threats to validity

We know that this work is subject to a variety of threats.

In particular, as stated in Section 7, we are aware that a different set of tools could be used to achieve the same results. The reason behind the choice of this specific set of tools is mainly based on the decision to create a free open-source pipeline that does not use any end-to-end tool but only relies on tools that are highly compatible and each of which partially covers the entire MLOps pipeline.

The results could have been influenced by the length of the different data frames used to simulate the CT setup. We know that

in an ML-based system one, the result is heavily based on the dataset used to represent the information and therefore used to perform training. The reason behind the selection of such a specific length relies on the decision of having a time frame that is informative for a big part of the lifetime of projects.

8 CONCLUSION

In this work, we presented an approach to develop an MLOps pipeline. Specifically by making use of OSSARA model guidelines, we created a model that performs continuous training on the available dataset and automatically deploys the best model to perform OSS risk assessment analysis. The integration of OSS tools such as DugsHub, DVC, and MLflow allowed the development of a fully automated system capable of keeping track of the different data, models, and experiments.

The use case has been used to describe what is the process of creating an MLOps pipeline and discussing what are the fundamental steps to take into account when creating an MLOps system. In the specific the OSSARA model developed using the MLOps pipeline has gained some new characteristics from this fruitful development. Some of these are continuous updates through continuous training, automatic deployment based on the model with the highest F1 score, and the possibility to deploy the model independently of the device through REST API.

Future works include the implementation of new pipelines to address different problems, including, among others, time series analysis, fault detection, and computer vision. Moreover, it would be useful to extend this in the context of continuous [15] and cognitive [14] edge-to-cloud.

REFERENCES

[1] Deepchecks AI. 2023. Continuous ML Validation - DeepChecks. https://deepchecks.com. (2023).

[2] Google Cloud Architecture Center. 2023. MLOps: Continuous delivery and automation pipelines in machine learning. https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning. (2023).

[3] Dagshub. 2023. Open Source Data Science Collaboration - DagsHub. https://dagshub.com. (2023).

[4] Digital.ai. 2023. Periodic Table of DevOps Tools. https://digital.ai/learn/devops-periodic-table/. (2023).

[5] Julian Ereth. 2018. DataOps-Towards a Definition. *LWDA* 2191 (2018), 104–112.

[6] IBM. 2023. What is AIOps? https://www.ibm.com/topics/aiops. (2023).

[7] IntelliJ IDEA. 2023. PyCharm: the Python IDE for Professional Developers by JetBrains. https://www.jetbrains.com/pycharm/. (2023).

[8] iterative.ai. 2023. Data Version Control - DVC. https://dvc.org. (2023).

[9] Ramtin Jabbari, Nauman bin Ali, Kai Petersen, and Binish Tanveer. 2016. What is DevOps? A systematic mapping study on definitions and practices. In *Proceedings of the Scientific Workshop Proceedings of XP2016*. 1–11.

[10] Valentina Lenarduzzi, Francesco Lomio, Sergio Moreschini, Davide Taibi, and Damian Andrew Tamburri. 2021. Software Quality for AI: Where We Are Now?. In *Software Quality: Future Perspectives on Software Engineering Quality*, Dietmar Winkler, Stefan Biffl, Daniel Mendez, Manuel Wimmer, and Johannes Bergsmann (Eds.). Springer International Publishing, Cham, 43–53.

[11] X. Li, S. Moreschini, F. Pecorelli, and D. Taibi. 2022. OSSARA: Abandonment Risk Assessment for Embedded Open Source Components. *IEEE Software* 39, 04 (jul 2022), 48–53. https://doi.org/10.1109/MS.2022.3163011

[12] Kiran Mainali, Lisa Ehrlinger, Mihhail Matskin, and Johannes Himmelbauer. 2021. Discovering DataOps: a comprehensive review of definitions, use cases, and tools. In *DATA ANALYTICS 2021 The Tenth International Conference on Data Analytics*.

[13] Sergio Moreschini, Francesco Lomio, David Hästbacka, and Davide Taibi. 2022. MLOps for evolvable AI intensive software systems. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 1293–1294. https://doi.org/10.1109/SANER53432.2022.00155

[14] Sergio Moreschini, Fabiano Pecorelli, Xiaozhou Li, Sonia Naz, Michele Albano, David Hästbacka, and Davide Taibi. 2023. Cognitive Cloud: The Definition. In *Distributed Computing and Artificial Intelligence, 19th International Conference*, Sigeru Omatu, Rashid Mehmood, Pawel Sitek, Serafino Cicerone, and Sara Rodríguez (Eds.). Springer International Publishing, Cham, 219–229.

[15] Sergio Moreschini, Fabiano Pecorelli, Xiaozhou Li, Sonia Naz, David Hästbacka, and Davide Taibi. 2022. Cloud Continuum: The Definition. *IEEE Access* 10 (2022), 131876–131886. https://doi.org/10.1109/ACCESS.2022.3229185

[16] Sergio Moreschini, Gilberto Recupito, Valentina Lenarduzzi, Fabio Palomba, David Hastbacka, and Davide Taibi. 2023. Toward End-to-End MLOps Tools Map: A Preliminary Study based on a Multivocal Literature Review. (2023). arXiv:cs.SE/2304.03254

[17] LF Projects. 2023. MLflow - A platform for the machine learning lifecycle. https://www.mlflow.org. (2023).

[18] Emmanuel Raj, David Buffoni, Magnus Westerlund, and Kimmo Ahola. 2021. Edge MLOps: An Automation Framework for AIoT Applications. In *2021 IEEE International Conference on Cloud Engineering (IC2E)*. 191–200. https://doi.org/10.1109/IC2E52221.2021.00034

[19] Philipp Ruf, Manav Madan, Christoph Reich, and Djaffar Ould-Abdeslam. 2021. Demystifying MLOps and Presenting a Recipe for the Selection of Open-Source Tools. *Applied Sciences* 11, 19 (2021). https://doi.org/10.3390/app11198861

[20] scikit learn. 2023. scikit-learn: machine learning in Python. https://scikit-learn.org/stable/. (2023).

[21] Rakshith Subramanya, Seppo Sierla, and Valeriy Vyatkin. 2022. From DevOps to MLOps: Overview and Application to Electricity Market Forecasting. *Applied Sciences* 12, 19 (2022). https://doi.org/10.3390/app12199851

[22] Damian A. Tamburri. 2020. Sustainable MLOps: Trends and Challenges. In *2020 22nd International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)*. 17–23. https://doi.org/10.1109/SYNASC51798.2020.00015

[23] Yue Zhou, Yue Yu, and Bo Ding. 2020. Towards MLOps: A Case Study of ML Pipeline Platform. In *2020 International Conference on Artificial Intelligence and Computer Engineering (ICAICE)*. 494–500. https://doi.org/10.1109/ICAICE51518.2020.00102